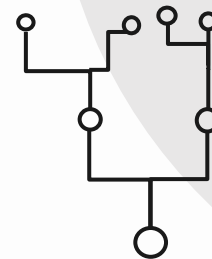


From Jupyter notebooks to workflow graphs

Reina Al Masri

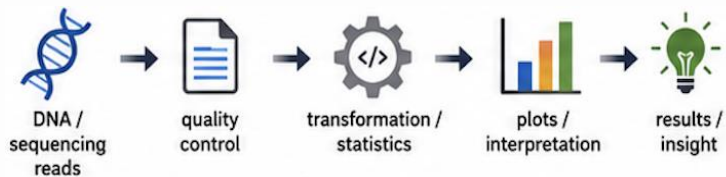
Supervised by George Marchment and Sarah Cohen-Boulakia

June 2026



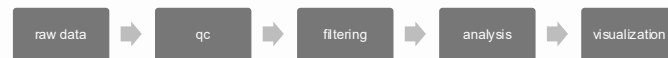
Context

Bioinformatics analysis



Bioinformatics uses code to analyse biological data through multiple connected steps

What is a workflow?



A workflow = an explicit chain of dependent analysis steps



Formal Workflow Systems



- Systems such as **Nextflow** make dependencies explicit
- **Bioflow-Insight** can statically reconstruct graph views, which **MetroFlow** displays as metro maps.



Problématique

The notebook problem

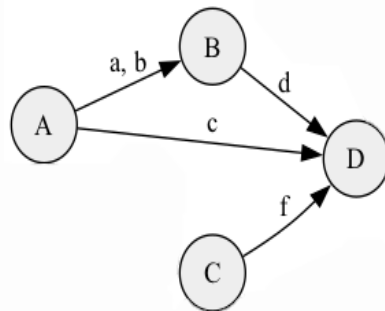
```
[ ]: # Cell A
a = 1
b = 1
c = 1

[ ]: # Cell B
d = a + b

[ ]: # Cell C
f = 2

[ ]: # Cell D
res = c + d + f
```

Static
analysis

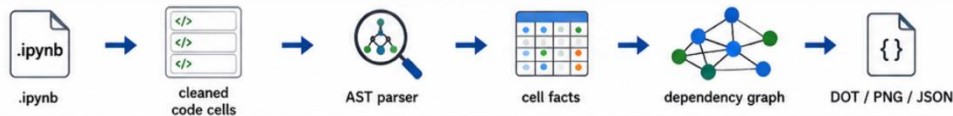


- Notebook order does not fully reveal computational structure
- A notebook cell may contain an entire analysis phase



Variables and functions may be defined in one cell and used much later

Prototype pipeline



Contribution: a static reconstruction of useful dataflow information from one notebook at a time, supporting visualization, comprehension, and integration with the BioFlow-Insight graph API.

Matériel et methods

Python's AST module



Why not plain text search?

A name like `df` for example could be appear in a comment, or a string, or an attribute, or a real variable access.



```
# df loaded
```

comment



```
"df"
```

string literal



```
obj.df
```

object attribute



```
print(df)
```

variable access

AST = code as a tree

Each node is one syntactic element:
Assignment, name, operation, function, call, etc.

Example

```
b = a + 1
```



Load = read

Store = defined

Résultats, analyse

Output types

```
[ ]: #Cell A
import random
def fun(x):
    return x+random.random()
```

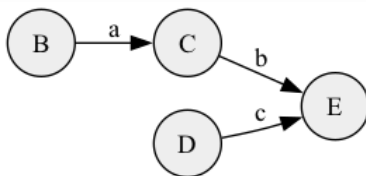
```
[ ]: #Cell B
a = 1
```

```
[ ]: #Cell C
b = fun(a)
```

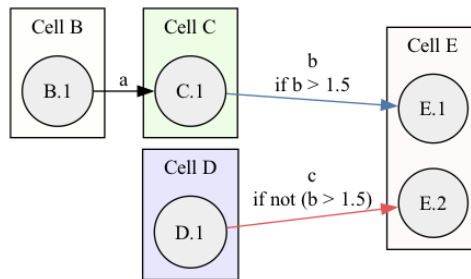
```
[ ]: #Cell D
c = 1
```

```
[ ]: #Cell E
if(b>1.5):
    res = fun(b)
else:
    res = fun(c)
```

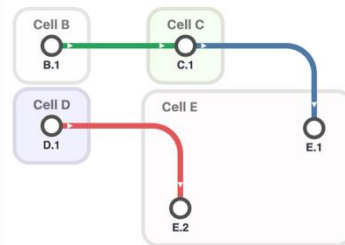
Cell-level graph



Statement-level graph
(expanded graph)



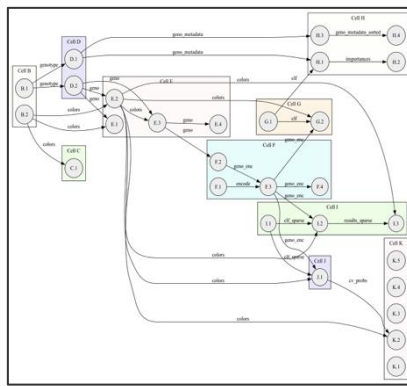
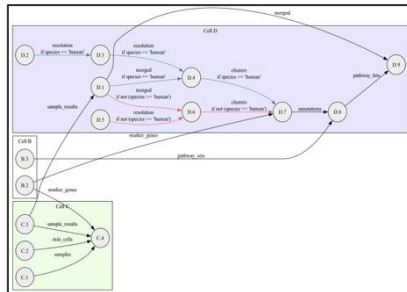
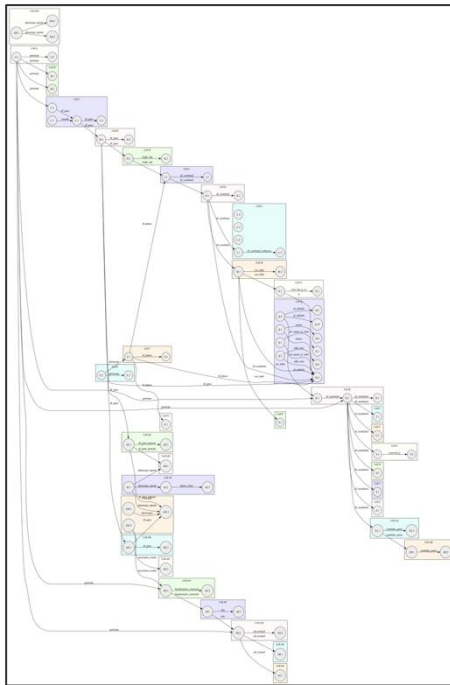
Metro map



Modeling rules

- Flat graph: cell = node
- Expanded graph: cell = box, statement = node
- $A \rightarrow B$ means B reads x, A last defined x
- Imports and function name alone do not create nodes
- function uses external x, $\rightarrow$ points to the call cell

Conclusion



Successfully implemented a parser that reconstructs dataflow from Jupyter notebooks, turning notebook code into cell-level and statement-level graphs, making hidden dependencies visible.

This extends the BioFlow-Insight approach from formal Nextflow workflows to widely used Jupyter notebooks.

What's next?

- Improve graph labels
- Handle more control flow: loops, **if** inside loops, **try/except**, **match**, **with**
- Go beyond one notebook: files, imports, other notebooks
- Add runtime information: execution order, hidden state, dynamic values